BOOLEAN ALGEBRA

Complete Exam-Oriented Study Guide

OAVS Computer Teacher | TGT Computer Science Preparation

Logic Gates · Truth Tables · Flip-Flops · Registers · Accumulators

Beginner-Friendly · Step-by-Step · Solved Examples · Practice MCQs

TABLE OF CONTENTS

SECTION 1	Introduction to Boolean Algebra	3
SECTION 2	Boolean Algebra Laws & Rules	4
SECTION 3	DeMorgan's Theorems	6
SECTION 4	Simplification of Boolean Equations	8
SECTION 5	Logic Gates (AND, OR, NOT, NAND, NOR, XOR, XNOR)	10
SECTION 6	Truth Tables – Construction & Examples	14
SECTION 7	Flip-Flops (SR, JK, D, T, Gated)	16
SECTION 8	Registers & Shift Registers	19
SECTION 9	Accumulators	21
SECTION 10	Exam Preparation – MCQs, Short & Long Q&A;	23
SECTION 11	Quick Revision & Formula Sheet	31

01 INTRODUCTION TO BOOLEAN ALGEBRA

What is Boolean Algebra?

Boolean Algebra is a branch of mathematics developed by **George Boole** in 1854. It deals with variables that have only **two possible values: 0 (False/OFF) and 1 (True/ON)**. It forms the mathematical foundation of digital electronics and computer science.

Importance in Digital Electronics & Computer Science

- Forms the basis of all digital circuit design (computers, mobile phones, calculators).
- Used to simplify complex logic circuits, saving cost and power.
- Helps in designing processors, memory units, and ALUs.
- Essential for programming logic and condition evaluation.
- Used in compiler design, database query optimization, and AI systems.

Binary Logic Basics

In Boolean Algebra, only **two logic levels** exist:

Logic Level	Value	Meaning	Voltage (TTL)
LOW	0	False / OFF / No	0 – 0.8 V
HIGH	1	True / ON / Yes	2 – 5 V

Boolean Variables, Constants & Operations

Variables: Represented by letters (A, B, C, X, Y, Z) — each can be 0 or 1.

Constants: Fixed values — either 0 or 1.

Three basic operations:

Operation	Symbol	Example	Meaning
AND	· or no symbol	A · B or AB	Both must be 1
OR	+	A + B	At least one must be 1
NOT	' or overline	A' or NOT A	Complement / Inversion

★ EXAM TIP

Boolean Algebra uses only 0 and 1. It is NOT ordinary algebra. 1 + 1 = 1 in Boolean (not 2). Always remember this in exams!

02 BOOLEAN ALGEBRA LAWS & RULES

Boolean Algebra follows a set of laws that allow simplification of expressions. Mastering these laws is essential for circuit design and exam success.

1. Identity Law

A + 0 = A	OR with 0 gives the variable itself
A · 1 = A	AND with 1 gives the variable itself

2. Null (Dominance) Law

A + 1 = 1	OR with 1 always gives 1
A · 0 = 0	AND with 0 always gives 0

3. Idempotent Law

A + A = A	OR of same variable
A · A = A	AND of same variable

4. Complement Law

A + A' = 1	A OR its complement = 1
A · A' = 0	A AND its complement = 0

5. Involution (Double Complement) Law

(A')' = A	Double NOT gives original variable

6. Commutative Law

A + B = B + A	Order doesn't matter in OR
A · B = B · A	Order doesn't matter in AND

7. Associative Law

(A+B)+C = A+(B+C)	Grouping doesn't matter in OR

(A·B)·C = A·(B·C)	Grouping doesn't matter in AND

8. Distributive Law

A·(B+C) = A·B + A·C	AND distributes over OR
A+(B·C) = (A+B)·(A+C)	OR distributes over AND

9. Absorption Law

A + A·B = A	A absorbs A·B
A · (A+B) = A	A absorbs A+B

10. Consensus Theorem

A·B + A'·C + B·C = A·B + A'·C	B·C is redundant term

Quick Reference – All Laws at a Glance

Law	OR Form	AND Form
Identity	A + 0 = A	A · 1 = A
Null	A + 1 = 1	A · 0 = 0
Idempotent	A + A = A	A · A = A
Complement	A + A' = 1	A · A' = 0
Involution	(A')' = A	(A')' = A
Commutative	A+B = B+A	AB = BA
Associative	(A+B)+C=A+(B+C)	(AB)C=A(BC)
Distributive	A+(BC)=(A+B)(A+C)	A(B+C)=AB+AC
Absorption	A + AB = A	A(A+B) = A

03 DeMORGAN'S THEOREMS

DeMorgan's Theorems are the most important rules in Boolean Algebra for simplification and converting between NAND/NOR expressions. Proposed by **Augustus De Morgan** (1847).

First DeMorgan's Theorem

Statement: The complement of a SUM equals the PRODUCT of the complements.

$$(A + B)' = A' \cdot B'$$

Proof using Truth Table:

A	B	A+B	(A+B)'	A'	B'	A'·B'
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

*Columns (A+B)' and A'·B' are **identical** — theorem proved! ✓*

Second DeMorgan's Theorem

Statement: The complement of a PRODUCT equals the SUM of the complements.

$$(A \cdot B)' = A' + B'$$

Proof using Truth Table:

A	B	A·B	(A·B)'	A'	B'	A'+B'
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

*Columns (A·B)' and A'+B' are **identical** — theorem proved! ✓*

Memory Trick for DeMorgan's Theorems

Break the bar, change the operator	• (A+B)' → A'·B' (break bar → + becomes ·) • (A·B)' → A'+B' (break bar → · becomes +)

Solved Examples – DeMorgan's Simplification

Example 1: Simplify: (A + B + C)'

Step 1: Apply 1st DeMorgan: (A+B+C)' = A'·B'·C'

■ **Answer: A'·B'·C'**

Example 2: Simplify: (A·B·C)'

Step 1: Apply 2nd DeMorgan: (A·B·C)' = A'+B'+C'

■ **Answer: A' + B' + C'**

Example 3: Simplify: ((A+B)·C)'

Step 1: Apply 2nd DeMorgan to whole: (A+B)' + C'

Step 2: Apply 1st DeMorgan to (A+B)': A'·B' + C'

■ **Answer: A'·B' + C'**

Example 4: Simplify: (A·B + C·D)'

Step 1: Apply 1st DeMorgan: (A·B)' · (C·D)'

Step 2: Apply 2nd DeMorgan to each: (A'+B')·(C'+D')

■ **Answer: (A'+B')·(C'+D')**

★ EXAM TIP

DeMorgan's is tested heavily. Remember: BREAK THE BAR, CHANGE THE OPERATOR. NOR gate implements 1st theorem; NAND gate implements 2nd theorem.

04 SIMPLIFICATION OF BOOLEAN EQUATIONS

Simplification reduces complex Boolean expressions to simpler forms, resulting in fewer gates and lower cost in hardware implementation.

SOP and POS Forms

SOP – Sum of Products:

An expression written as OR of AND terms. Each AND term is called a **minterm**.

Example: F = AB + A'C + BC

POS – Product of Sums:

An expression written as AND of OR terms. Each OR term is called a **maxterm**.

Example: F = (A+B)(A'+C)(B+C)

Step-by-Step Algebraic Simplification

Example 1: Simplify F = AB + AB'

→ AB + AB' [Factor out A]

→ = A(B + B') [Complement Law: B+B'=1]

→ = A · 1 [Identity Law: A·1=A]

→ = A

■ **Answer: F = A**

Example 2: Simplify F = A + A'B

→ A + A'B [Distributive: A = A·1 = A(B+B')]

→ = AB + AB' + A'B [Group: AB' stays, factor A'B+AB]

→ = A + A'B → Apply Absorption variant

→ = (A + A')(A + B) [Distributive: A+(BC)=(A+B)(A+C)]

→ = 1·(A+B) [Complement: A+A'=1]

→ = A + B

■ **Answer: F = A + B**

Example 3: Simplify F = ABC + ABC' + AB'C + A'BC

→ Group first two: AB(C+C') = AB

→ AB + AB'C + A'BC

→ Group AB + AB'C: A(B + B'C) = A(B+C) [Absorption variant]

→ = AB + AC + A'BC

→ = AB + C(A + A'B) = AB + C(A+B) [from Ex.2]

→ = AB + AC + BC

■ Answer: F = AB + AC + BC

★ EXAM TIP

For exam: focus on algebraic simplification using Boolean laws. Practice applying Identity, Complement, Absorption and Distributive laws step by step.

05 LOGIC GATES

Logic gates are the fundamental building blocks of digital circuits. They perform Boolean operations on one or more binary inputs to produce a single binary output.

AND Gate

Y = A · B (also written Y = AB)

Output is HIGH (1) only when ALL inputs are HIGH (1). Works like multiplication: 0×anything=0, 1×1=1.

A	B	Y = A·B
0	0	0
0	1	0
1	0	0
1	1	1

Real-life Applications:

- Safety interlocks (alarm sounds only when door AND motion sensor active)
- Enable/disable circuits
- Digital multiplication circuits

★ EXAM TIP

AND = Multiplication. Output 1 only when A=1 AND B=1.

OR Gate

Y = A + B

Output is HIGH (1) when ANY input is HIGH (1). Works like addition (with 1+1=1 in Boolean).

A	B	Y = A+B
0	0	0
0	1	1
1	0	1
1	1	1

Real-life Applications:

- Alarm systems (alarm if any sensor triggers)
- Input selection circuits
- Error detection (if any error bit is 1)

★ EXAM TIP

OR = Addition (Boolean). Output 0 only when ALL inputs are 0.

NOT Gate (Inverter)

Y = A' (also written Y = ■)

Single input gate. Output is always the COMPLEMENT (opposite) of input. If input is 0, output is 1 and vice versa.

A	Y = A'
0	1
1	0

Real-life Applications:

- Signal inversion
- Creating complements for other operations
- Oscillators and clock circuits

★ EXAM TIP

NOT inverts the signal. Also called Inverter. Only 1 input, 1 output.

NAND Gate

Y = (A · B)'

NAND = NOT + AND. Output is LOW (0) ONLY when ALL inputs are HIGH (1). Called the **Universal Gate** — can implement ANY Boolean function.

A	B	A·B	Y=(A·B)'
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

Real-life Applications:

- Universal gate — used in all IC families
- TTL and CMOS digital circuits
- Memory cells and latches

★ EXAM TIP

NAND is Universal Gate. NAND of same input = NOT. Most widely used gate in ICs.

NOR Gate

Y = (A + B)'

NOR = NOT + OR. Output is HIGH (1) ONLY when ALL inputs are LOW (0). Also a **Universal Gate** — can implement any function using NOR only.

A	B	A+B	Y=(A+B)'
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

Real-life Applications:

- Universal gate
- CMOS logic families
- Flip-flop circuits

★ EXAM TIP

NOR is also a Universal Gate. NOR of same input = NOT gate.

XOR Gate (Exclusive OR)

Y = A ⊕ B = A'B + AB'

Output is HIGH (1) when inputs are DIFFERENT. Output is LOW (0) when inputs are SAME. Used extensively in adder circuits.

A	B	Y = A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

Real-life Applications:

- Half adder and full adder circuits
- Parity checkers and generators
- Error detection codes (CRC)
- Comparator circuits

★ EXAM TIP

XOR: output 1 when inputs DIFFER. output 0 when inputs SAME. Used in adders!

XNOR Gate (Exclusive NOR)

Y = (A ⊕ B)' = AB + A'B'

Complement of XOR. Output is HIGH (1) when inputs are SAME. Output is LOW (0) when inputs are DIFFERENT. Also called **Equality Gate**.

A	B	Y = (A⊕B)'
0	0	1
0	1	0
1	0	0
1	1	1

Real-life Applications:

- Digital comparators (check if A=B)
- Error detection
- Parity checking circuits

★ EXAM TIP

XNOR: output 1 when inputs SAME. Called Equality Gate. Complement of XOR.

Logic Gates – Complete Comparison Table

Gate	Symbol	Expression	Output=1 when...	Universal ?
AND	&	AB	All inputs = 1	No
OR	≥1	A+B	Any input = 1	No
NOT	1	A'	Input = 0	No
NAND	& ■	(AB)'	Not all inputs = 1	**YES**
NOR	≥1 ■	(A+B)'	All inputs = 0	**YES**
XOR	=1	A⊕B	Inputs are different	No
XNOR	=1 ■	(A⊕B)'	Inputs are same	No

06 TRUTH TABLES – CONSTRUCTION & EXAMPLES

A Truth Table lists ALL possible input combinations and the corresponding output for a Boolean expression. It is the most reliable method to verify Boolean equations.

Rules for Constructing Truth Tables

1. For n input variables, there are 2^n rows (combinations).
2. Columns: list all input variables, intermediate expressions, then final output.
3. Fill inputs in binary order (000...0 to 111...1).
4. Evaluate each intermediate expression left to right.
5. Use the order of precedence: NOT > AND > OR.

Example 1: F = A·B + A'·C (3 variables)

A	B	C	A'	A·B	A'·C	F = A·B + A'·C
0	0	0	1	0	0	0
0	0	1	1	0	1	1
0	1	0	1	0	0	0
0	1	1	1	0	1	1
1	0	0	0	0	0	0
1	0	1	0	0	0	0
1	1	0	0	1	0	1
1	1	1	0	1	0	1

Example 2: F = (A + B)·(A' + C)

A	B	C	A+B	A'	A'+C	F
0	0	0	0	1	1	0
0	0	1	0	1	1	0
0	1	0	1	1	1	1
0	1	1	1	1	1	1
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	0	1	0	0	0
1	1	1	1	0	1	1

Example 3: F = A ⊕ B ⊕ C (XOR of 3 inputs)

A	B	C	A⊕B	F = A⊕B⊕C
0	0	0	0	0
0	0	1	0	1
0	1	0	1	1
0	1	1	1	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	0	1

*Note: A⊕B⊕C = 1 when an **ODD** number of inputs are 1. This is the basis of parity bit generation!*

★ EXAM TIP

Number of rows = 2^n where n = number of input variables. For 2 inputs: 4 rows; 3 inputs: 8 rows; 4 inputs: 16 rows.

07 FLIP-FLOPS

A **Flip-Flop** is a bistable sequential logic circuit that stores 1 bit of information. It has two stable states (0 or 1) and changes state based on inputs and clock signal. Flip-flops are the basic building blocks of registers, counters, and memory.

Types of Flip-Flops

Type	Full Name	Inputs	Key Feature
SR Flip-Flop	Set-Reset	S, R	Most basic. S=Set output to 1; R=Reset to 0.
JK Flip-Flop	Jack-Kilby	J, K	Improved SR. Eliminates invalid state.
D Flip-Flop	Data/Delay	D	Stores 1 bit; output follows input on clock.
T Flip-Flop	Toggle	T	Toggles state when T=1; holds when T=0.

SR Flip-Flop (Set-Reset Flip-Flop)

The SR Flip-Flop has two inputs: **S (Set)** and **R (Reset)**, two outputs: **Q** (normal) and **Q'** (complement).

S=1, R=0 → Q=1 (Set); S=0, R=1 → Q=0 (Reset); S=0, R=0 → Q unchanged; S=1, R=1 → INVALID

Characteristic Table:

S	R	Q (next)	Action
0	0	Q	No Change (Hold)
0	1	0	Reset (Q=0)
1	0	1	Set (Q=1)
1	1	X	INVALID (Forbidden!)

■ WARNING

S=1 R=1 is FORBIDDEN in SR flip-flop — it creates an unpredictable state. This is the main disadvantage. JK flip-flop fixes this!

JK Flip-Flop

The JK Flip-Flop is an **improved version of SR**. When J=1, K=1, the output **toggles** (complements) instead of being invalid. J acts like S (Set) and K acts like R (Reset).

Characteristic Table:

J	K	Q (next)	Action
0	0	Q	No Change

0	1	0	Reset
1	0	1	Set
1	1	Q'	Toggle (Complement)

Characteristic Equation: Q(next) = JQ' + K'Q

D Flip-Flop (Data/Delay Flip-Flop)

The D Flip-Flop has a single input **D**. The output Q follows D after each clock pulse. It is used for **data storage and shift registers**. Eliminates invalid state completely.

D	Q (next)	Action
0	0	Reset (Q follows D)
1	1	Set (Q follows D)

Characteristic Equation: Q(next) = D

T Flip-Flop (Toggle Flip-Flop)

The T Flip-Flop has a single input **T**. When T=0, output holds; when T=1, output toggles. Used in **counters** and frequency dividers.

T	Q (next)	Action
0	Q	No Change (Hold)
1	Q'	Toggle

Characteristic Equation: Q(next) = T⊕Q = TQ' + T'Q

Gated Flip-Flops (Clocked Flip-Flops)

A **Gated Flip-Flop** (or Clocked Flip-Flop) changes state only when a **clock (CLK) signal** is active. This prevents unwanted state changes and enables synchronous operation.

Type	Trigger Condition	Feature
Level-Triggered	State changes when CLK is HIGH (1) or LOW (0)	Simpler; may cause glitches
Edge-Triggered	State changes on Rising Edge (0→1) or Falling Edge (1→0) of CLK	More reliable; used in practice
Master-Slave	Two flip-flops connected; captures input on one edge, outputs on other	Eliminates race condition

Flip-Flop Comparison at a Glance

Property	SR	JK	D	T
# Inputs	2 (S,R)	2 (J,K)	1 (D)	1 (T)
Invalid State?	Yes (S=R=1)	No	No	No
Toggle Feature?	No	Yes	No	Yes
Equation	—	JQ'+K'Q	D	T⊕Q
Primary Use	Latch	Counter	Register	Counter

08 REGISTERS & SHIFT REGISTERS

A **Register** is a group of flip-flops connected together to store a group of binary bits (a word). An n-bit register uses n flip-flops and can store n bits of data simultaneously.

Types of Registers

Storage Register: Stores binary data; loaded and read in parallel

Shift Register: Bits can be shifted left or right on each clock pulse

Buffer Register: Temporarily holds data between two systems of different speeds

Index Register: Used in CPUs for address modification

Shift Registers – Four Types

Shift registers are classified by how data enters and exits:

SISO – Serial In Serial Out

Data enters one bit at a time (serially) and exits one bit at a time (serially). Takes n clock pulses to load n bits and n more to read them out.

Example: Load 1011: feed 1, then 0, then 1, then 1 bit-by-bit; output appears one bit at a time after n clocks.

Uses: Data transmission over single wire, time delay circuits

SIPO – Serial In Parallel Out

Data enters serially (1 bit at a time) but all bits are available simultaneously at the output after n clock pulses.

Example: Load 1011 serially → After 4 clocks, Q3Q2Q1Q0 = 1011 all at once.

Uses: Serial-to-parallel data conversion, communication interfaces

PISO – Parallel In Serial Out

All bits are loaded simultaneously (in parallel) and shifted out one bit at a time.

Example: Load 1011 in parallel → Shift out: 1, 0, 1, 1 on successive clocks.

Uses: Parallel-to-serial conversion, keyboard data transmission

PIPO – Parallel In Parallel Out

Data is loaded simultaneously and read simultaneously. Fastest type — only 1 clock pulse needed.

Example: Load 1011 → Immediately read Q3Q2Q1Q0 = 1011.

Uses: CPU registers, buffer storage, fast data transfer

Shift Register Comparison Table

Type	Input Mode	Output Mode	Clocks to Load	Clocks to Read

SISO	Serial	Serial	n clocks	n clocks
SIPO	Serial	Parallel	n clocks	1 clock
PISO	Parallel	Serial	1 clock	n clocks
PIPO	Parallel	Parallel	1 clock	1 clock

Applications of Registers

- Store intermediate results in CPU (general-purpose registers: AX, BX, CX, DX)
- Serial-to-parallel and parallel-to-serial data conversion
- Multiplication and division by powers of 2 (left shift = ×2, right shift = ÷2)
- Time delay circuits in signal processing
- Ring counters and Johnson counters
- Data transfer between peripherals and CPU

★ EXAM TIP

Left shift by 1 position = Multiply by 2. Right shift by 1 = Divide by 2. PIPO is fastest; SISO is slowest. 4-bit register stores values 0–15.

09 ACCUMULATORS

An **Accumulator** is a special-purpose register in the CPU's ALU (Arithmetic Logic Unit) that stores the intermediate results of arithmetic and logic operations.

Definition & Working Principle

The accumulator is designed to **accumulate** (collect/store) results of successive operations. In older architectures (like 8085, 8086), one operand always came from/went to the accumulator.

Step 1: Load first operand into accumulator (e.g., A = 5)

Step 2: ALU performs operation (e.g., A + 3)

Step 3: Result stored back in accumulator (A = 8)

Step 4: Next operation uses current accumulator value

Step 5: Final result can be stored in memory or output

Accumulator vs General Register

Feature	Accumulator	General Register
Purpose	Stores ALU results	General storage
Connection	Directly wired to ALU	Via data bus
Usage	Automatic with every operation	Must be specified in instruction
Speed	Fastest (direct ALU access)	Slightly slower
Count in CPU	Usually 1 (accumulator)	Multiple (AX, BX, CX, DX...)
Examples	A register in 8085/8086	BX, CX, DX in x86

Real-World Example: 8085 Microprocessor

In the Intel 8085 microprocessor, the **A register** (8-bit) is the accumulator. All arithmetic operations (ADD, SUB, AND, OR, XOR) store their results in A.

Instruction	Operation	Accumulator Value
MVI A, 05H	Load 05 (hex) into A	A = 0000 0101
ADD B	A = A + B (assume B=03)	A = 0000 1000
ANI 0FH	A = A AND 0F (mask)	A = 0000 1000
STA 2000H	Store A to memory 2000H	A unchanged = 08H

★ EXAM TIP

Accumulator = A register in 8085. It is part of the ALU. Every arithmetic/logic operation result goes to the accumulator automatically.

10 EXAM PREPARATION

Important Formulas & Quick Reference

Basic Laws:

- A+0=A, A·1=A (Identity)
- A+1=1, A·0=0 (Null)
- A+A=A, A·A=A (Idempotent)
- A+A'=1, A·A'=0 (Complement)
- (A')'=A (Involution)

DeMorgan's:

- (A+B)' = A'·B' (1st Theorem)
- (A·B)' = A'+B' (2nd Theorem)
- (A+B+C)' = A'·B'·C'
- (A·B·C)' = A'+B'+C'

Gate Expressions:

- AND: Y=AB
- OR: Y=A+B
- NOT: Y=A'
- NAND: Y=(AB)'
- NOR: Y=(A+B)'
- XOR: Y=A⊕B=A'B+AB'
- XNOR: Y=(A⊕B)'=AB+A'B'

Flip-Flop Equations:

- SR: Q(n+1)=S+R'Q (R'S=0 condition)
- JK: Q(n+1)=JQ'+K'Q
- D: Q(n+1)=D
- T: Q(n+1)=T⊕Q=TQ'+T'Q

Multiple Choice Questions (MCQs)

1. Which gate is called the Universal Gate?

(a) AND

(b) OR

(c) NAND

(d) XOR

■ **Answer: (c)**

2. DeMorgan's First Theorem states:

(a) (AB)' = A'+B'

(b) (A+B)' = A'·B'

(c) A+A'=1

(d) A·A=A

■ **Answer: (b)**

3. What is the output of XOR gate when A=1, B=1?

(a) 0

(b) 1

(c) X

(d) A'

■ **Answer: (a)**

4. How many rows are in a truth table with 4 input variables?

(a) 4

(b) 8

(c) 12

(d) 16

■ **Answer: (d)**

5. Which flip-flop has the forbidden state when J=K=1?

(a) SR

(b) JK

(c) D

(d) T

■ **Answer: (a)**

6. The characteristic equation of JK flip-flop is:

(a) Q=D

(b) Q=T⊕Q

(c) Q=JQ'+K'Q

(d) Q=S+RQ

■ **Answer: (c)**

7. Left shift of a binary number by 1 position is equivalent to:

(a) Dividing by 2

(b) Multiplying by 2

(c) Adding 1

(d) Subtracting 1

■ **Answer: (b)**

8. SIPO register stands for:

(a) Serial In Parallel Out

(b) Serial In Parallel Over

(c) Shift In Parallel Out

(d) None

■ **Answer: (a)**

9. The complement of A+B using DeMorgan's is:

(a) A'+B'

(b) A'·B'

(c) A+B

(d) A·B

■ **Answer: (b)**

10. Accumulator is part of which unit of CPU?

(a) CU

(b) Memory

(c) ALU

(d) I/O unit

■ **Answer: (c)**

11. Which law states A + AB = A?

(a) Distributive

(b) Idempotent

(c) Absorption

(d) Null

■ **Answer: (c)**

12. NOR gate is universal because:

(a) It has two inputs

(b) Any Boolean function can be implemented using NOR alone

(c) Its output is always 1

(d) It is fastest

■ **Answer: (b)**

13. How many flip-flops are needed for a 8-bit register?

(a) 4

(b) 6

(c) 8

(d) 16

■ **Answer: (c)**

14. Which flip-flop has Q(next) = D?

(a) SR

(b) T

(c) JK

(d) D

Answer: (d)

15. XNOR gate output is 1 when:

(a) Inputs are different

(b) Inputs are same

(c) All inputs are 0

(d) Any input is 1

Answer: (b)

True / False Questions

1. NAND gate is a Universal Gate.

■ **True**

2. In Boolean Algebra, A + 1 = A.

■ **False (A+1 = 1)**

3. XOR gate output is 1 when inputs are same.

■ **False (output is 0)**

4. JK flip-flop eliminates the invalid state of SR flip-flop.

■ **True**

5. Left shift multiplies a binary number by 2.

■ **True**

6. D flip-flop has two inputs.

■ **False (it has only one input D)**

7. The complement of (A+B) is A'+B'.

■ **False — it is A'·B' by DeMorgan's**

8. PIPO register requires only 1 clock pulse to load and read.

■ **True**

9. Accumulator stores the final output of the program.

■ **False — it stores intermediate results**

10. Boolean 1 + 1 = 2.

■ **False — 1 + 1 = 1 in Boolean Algebra**

Fill in the Blanks

1. (A·B)' = _____ (DeMorgan's)

■ **Answer: A' + B'**

2. The Boolean expression for XNOR gate is _____.

■ **Answer: AB + A'B'**

3. A register that shifts bits left or right is called _____.

■ **Answer: Shift Register**

4. The forbidden input condition of SR flip-flop is S = __ R = __.

■ **Answer: S=1, R=1**

5. Number of rows in truth table with 3 variables = _____.

■ **Answer: 8 (2^3)**

6. The flip-flop whose output toggles when T=1 is _____ flip-flop.

■ **Answer: T**

7. _____ is the main register of ALU that stores results.

■ **Answer: Accumulator**

8. A + A' = _____.

■ **Answer: 1**

9. The Universal gates are _____ and _____.

■ **Answer: NAND, NOR**

Short Answer Questions (2–3 marks)

Q1. State DeMorgan's two theorems.

1st: (A+B)' = A'·B' — Complement of sum = product of complements. 2nd: (A·B)' = A'+B' — Complement of product = sum of complements.

Q2. What is a Universal Gate? Name them.

A gate from which all other gates can be implemented is called a Universal Gate. NAND and NOR are universal gates.

Q3. Differentiate between XOR and XNOR gates.

XOR: Output=1 when inputs are DIFFERENT. Expression: A⊕B = A'B+AB'. XNOR: Output=1 when inputs are SAME. Expression: (A⊕B)' = AB+A'B'. XNOR is complement of XOR.

Q4. What is a Flip-Flop?

A flip-flop is a bistable sequential logic circuit that stores 1 bit of information. It has two stable states (0 and 1) and changes state based on clock and input signals.

Q5. Explain Absorption Law with example.

Absorption Law: A + AB = A (A absorbs AB). Proof: A + AB = A(1+B) = A·1 = A [using Null Law: 1+B=1]

Q6. What is K-Map and what is its use?

It arranges truth table values in a grid where adjacent cells differ by 1 bit. Groups of 1s give simplified expression terms.

Q7. What is the difference between SISO and PIPO?

SISO (Serial In Serial Out): Data loads and exits one bit at a time — needs n clocks each. PIPO (Parallel In Parallel Out): All bits load and exit simultaneously — only 1 clock needed. PIPO is fastest.

Long Answer Questions (5 marks)

Q1. Explain all logic gates with their truth tables, Boolean expressions, and one real-life application each.

Answer outline: Cover: AND, OR, NOT, NAND, NOR, XOR, XNOR with symbols, expressions, truth tables (see Section 5).

Q2. Explain the four types of flip-flops (SR, JK, D, T) with their characteristic tables.

Answer outline: Define each flip-flop, list inputs/outputs, give characteristic table, mention equation, compare (see Section 7).

Q3. Simplify F = A'BC + AB'C + ABC' + ABC using Boolean algebra.

Answer outline: Step 1: Group ABC'+ABC = AB(C'+C) = AB Step 2: Group AB'C+ABC = AC(B'+B) = AC Step 3: Group A'BC — stands alone initially Step 4: F = AB + AC + A'BC Step 5: AB + AC + A'BC (apply AB+A'BC = AB+A'C from Consensus: AB+A'BC = AB+A'C) ■ F = AB + AC + A'C = AB + C(A+A') = AB + C = AB + C Actually: F = AB + A'C + AC = AB + C(A+A') = AB + C ■ F = AB + C

Q4. Explain the four types of shift registers with diagrams and applications.

Answer outline: SISO, SIPO, PISO, PIPO — explain input/output mode, clock requirements, data flow, uses (see Section 8).

Q5. Prove DeMorgan's theorems using truth tables.

Answer outline: For both (A+B)'=A'B' and (A·B)'=A'+B', construct 4-row truth tables and show columns match (see Section 3).

11 QUICK REVISION & FORMULA SHEET

One-Line Revision Notes

01. Boolean Algebra has only two values: 0 and 1.
02. AND = multiplication; OR = addition (Boolean); NOT = complement.
03. A+1=1 (not A+1=A!); A·0=0; A+A'=1; A·A'=0.
04. DeMorgan's: Break the bar, change the operator.
05. NAND and NOR are Universal Gates.
06. XOR=1 when inputs DIFFER; XNOR=1 when inputs SAME.
07. SR flip-flop: S=1,R=1 is FORBIDDEN.
08. JK flip-flop eliminates invalid state; toggles when J=K=1.
09. D flip-flop: Q(next)=D (simply stores D).
10. T flip-flop: Q(next)=T⊕Q (toggles when T=1).
11. Shift left = ×2; Shift right = ÷2.
12. PIPO fastest; SISO slowest for data loading.
13. Accumulator = part of ALU; stores intermediate results.
14. n-input truth table has 2^n rows.
15. SOP = Sum of Products (OR of ANDs); POS = Product of Sums (AND of ORs).

Master Summary Table – Boolean Algebra

Topic	Key Points	Remember
Boolean Algebra	2 values: 0 and 1	1+1=1 (not 2!)
AND Gate	Y=AB; output 1 if all=1	Multiplication
OR Gate	Y=A+B; output 1 if any=1	Addition
NAND Gate	Y=(AB)'; Universal gate	Universal #1
NOR Gate	Y=(A+B)'; Universal gate	Universal #2
XOR Gate	Y=A⊕B; output 1 if different	Adder circuits
XNOR Gate	Y=(A⊕B)'; output 1 if same	Equality gate
DeMorgan's 1st	(A+B)' = A'B'	Break bar, change +→·
DeMorgan's 2nd	(A·B)' = A'+B'	Break bar, change ·→+
SR Flip-Flop	S=Set, R=Reset; S=R=1 forbidden	Oldest type
JK Flip-Flop	J=K=1 → Toggle; no invalid	Improved SR
D Flip-Flop	Q=D; 1 input; simplest	Data storage
T Flip-Flop	T=1→toggle; T=0→hold	Counters

Shift Register	SISO,SIPO,PISO,PIPO	PIPO=fastest
Accumulator	ALU register; stores results	A reg in 8085

■ BEST WISHES

Best of luck for your OAVS / TGT Computer Science exam! Revise DeMorgan's theorems, gate truth tables, and flip-flop equations daily. Practice simplification problems usingalgebraic Boolean laws.

Complete Study Guide